

使用 Gemini、Veo 和 Cloud Run 建構及部署 AI Motion Lab

來源：<https://codelabs.developers.google.com/codelabs/gemini-motion-lab/instructions?hl=zh-tw>

步驟數：9

部署即時 AI 資訊站體驗，使用 Gemini 分析人體動作、透過 Nano Banana 生成風格化虛擬化身、使用 Veo 製作 AI 影片，並完全在 Cloud Run 上執行。

目錄

1. [1. 簡介](#)
2. [2. 📦 複製存放區](#)
3. [3. 🔧 申請抵免額並建立 GCP 專案](#)
4. [4. 🧠 \[READ ONLY\] Understanding the Architecture](#)
5. [5. ⚙️ 執行設定指令碼](#)
6. [6. 🚀 部署後端](#)
7. [7. 🌈 部署前端](#)
8. [8. 🎮 \[選用\] 試玩試用版](#)
9. [9. 🎉 結論](#)

1. 簡介

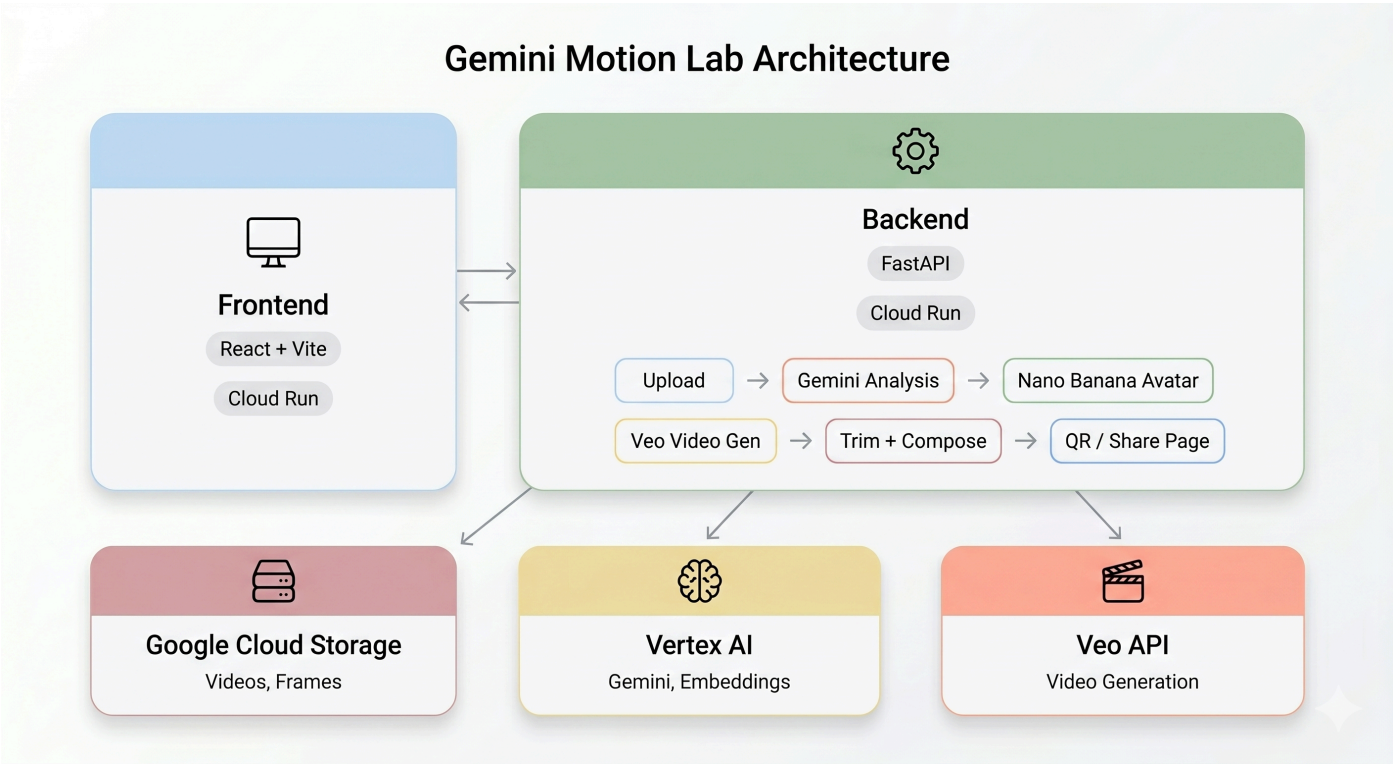
建構項目

Gemini Motion Lab 是由 AI 驅動的現場互動式資訊站，使用者錄製一段短片，內容為舞蹈或動作，系統會：

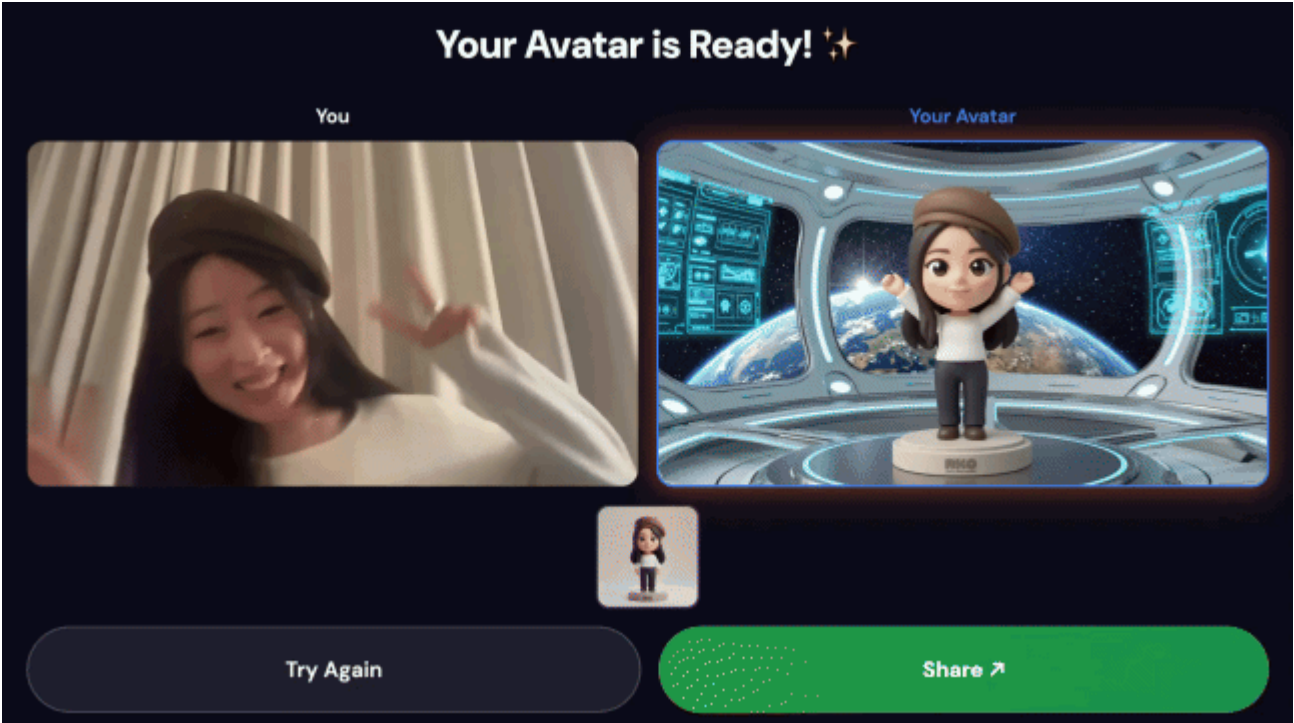
- 1. 使用 **Gemini** 分析動作 (身體部位、階段、節奏、能量)
- 2. 使用 **Nano Banana** (Gemini Flash Image) 生成風格化的顯示圖片
- 3. 使用 **Veo** 製作 AI 影片，讓虛擬化身重現動作
- 4. 製作並排影片 (原始影片 + AI 生成影片)
- 5. 分享結果：透過 QR code 在針對行動裝置最佳化網頁上分享結果

完成本程式碼研究室後，您將在 **Google Cloud Run** 中部署完整範例，並瞭解支援範例的 AI 管道。

架構總覽



最終版試用：



核心技術

元件	科技	目的
動作分析	Gemini Flash	分析影片中的身體動作、階段和風格
生成虛擬化身	Gemini Flash Image (Nano Banana)	從主要畫面格生成風格化 1024×1024 虛擬化身
影片生成	Veo 3.1	根據虛擬化身和動作提示詞製作 AI 影片
後端	FastAPI + Python 3.11	API 伺服器，可自動調度非同步管道
前端	React + Vite + TypeScript	資訊站使用者介面，顯示攝影機錄影和即時狀態
代管	Cloud Run	無伺服器容器化部署
儲存空間	Google Cloud Storage	上傳影片、影格、剪輯和合成輸出內容

步驟 2 · 約 0 分鐘

2. 📦 複製存放區

1. 開啟 Cloud Shell 編輯器

👉 在瀏覽器中開啟 [Cloud Shell 編輯器](#)。

如果畫面底部未顯示終端機：

- 按一下「查看」
- 按一下「終端機」。

2. 複製程式碼

👉 🖨 在終端機中複製存放區：

```
cd ~
git clone https://github.com/cuppibla/gemini-motion-lab-starter.git
cd gemini-motion-lab-starter
```

3. 探索專案結構

快速瀏覽存放區版面配置：

```
gemini-motion-lab-starter/
├── backend/                                # FastAPI backend (Python 3.11)
│   ├── app/
│   │   ├── main.py                        # FastAPI app entry point
│   │   ├── config.py                     # Environment-based settings
│   │   ├── routers/                      # API endpoints (upload, analyze, generate, share...)
│   │   ├── services/                     # Business logic (Gemini, Veo, storage, pipeline...)
│   │   └── prompts/                      # AI prompt templates
│   ├── Dockerfile
│   └── pyproject.toml
├── frontend/                              # React + Vite + TypeScript
│   ├── src/                              # React components
│   ├── public/                           # Static assets
│   ├── Dockerfile
│   └── nginx.conf
├── init.sh                               # Create GCP project & link billing
├── billing-enablement.py                 # Auto-link billing account
├── setup.sh                             # Create GCS bucket, service account, .env
└── scripts/                             # Utility scripts
```

重要目錄：

- `backend/app/services/` - 這是 AI 魔法的發生地 (Gemini、Nano Banana、Veo)
 - `backend/app/routers/` - 前端呼叫的 API 端點
 - `backend/app/prompts/`：精心設計的提示，用於動作分析、生成虛擬化身和影片
-

步驟 3 · 約 0 分鐘

3. 申請抵免額並建立 GCP 專案

第 1 部分：領取帳單抵免額

無須提供信用卡或付款方式，即可獲得免費抵免額。只要輸入姓名並接受抵免額，不到 2 分鐘即可完成。

 使用 **Gmail** 帳戶申請帳單帳戶抵免額。

第 2 部分：建立新專案

  在終端機中，將 `init` 指令碼設為可執行檔並執行：

```
cd ~/gemini-motion-lab-starter
chmod +x init.sh
./init.sh
```

 **專案 ID 注意事項：**指令碼會提示您建立新的 Google Cloud 專案。您可以接受系統隨機產生的預設專案 ID，也可以自行指定。

`init.sh` 指令碼會執行下列作業：

1. 建立新的 GCP 專案，並加上 `gemini-motion-lab` 前置字元
2. 將專案 ID 儲存至 `~/project_id.txt`
3. 安裝帳單依附元件，並自動連結帳單帳戶

第 3 部分：設定專案並啟用 API

  在終端機中設定專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

  啟用這個專案所需的 Google Cloud API (這需要約 1 到 2 分鐘)：

```
gcloud services enable \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  aiplatform.googleapis.com \  
  storage.googleapis.com \  
  artifactregistry.googleapis.com
```

各個 API 的用途：

API	目的
<code>run.googleapis.com</code>	Cloud Run ：以無伺服器容器的形式代管後端和前端
<code>cloudbuild.googleapis.com</code>	Cloud Build ：從原始碼建構 Docker 映像檔
<code>aiplatform.googleapis.com</code>	Vertex AI ：存取 Gemini 和 Veo 模型
<code>storage.googleapis.com</code>	Cloud Storage ：儲存上傳的影片、影格和產生的素材資源
<code>artifactregistry.googleapis.com</code>	Artifact Registry ：儲存建構的 Docker 映像檔

步驟 4 · 約 0 分鐘

4. 🧠 [READ ONLY] Understanding the Architecture

本節說明 AI 管道的端對端運作方式。無須採取任何行動，只要在部署前閱讀本文，瞭解系統即可。

AI 管道

使用者在資訊站錄製動態短片時，系統會依序執行五個階段：



第 1 階段：上傳影片

前端會從使用者的攝影機錄製 5 秒的 WebM 短片，並透過後端的 `/api/upload` 端點上傳至 Google Cloud Storage。

```
POST /api/upload/{video_id} → gs://BUCKET/uploads/{video_id}.webm
```

第 2 階段：Gemini 動作分析

後端會將上傳的影片傳送至 Gemini Flash (`gemini-3-flash-preview`) 進行結構化分析。

Gemini 擷取的內容： Gemini 會觀看影片，並傳回結構化 JSON，其中包含：

- `movement_summary`：動作的自然語言說明
- `body_parts` - 涉及的身體部位 (手臂、軀幹、腿部等)
- `phases` - 時間區隔細目 (日期範圍、動作、節奏、能量)
- `best_frame_timestamp`：擷取靜態畫面的最佳時刻
- `veo_prompt`：為 Veo 影片生成功能精心設計的提示
- `person_description`：生成虛擬化身時的個人描述

運作方式 (backend/app/services/gemini_service.py) :

這項服務會使用 Vertex AI SDK 的 `client.models.generate_content()`，並以影片做為 `Part.from_uri` 輸入內容和結構化提示。`response_mime_type="application/json"` 可確保 Gemini 傳回可剖析的 JSON。此外，模型也會使用 `ThinkingConfig(thinking_budget=1024)`，更準確地推論動作階段。

```
# Simplified from gemini_service.py
response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=[
        types.Part.from_uri(file_uri=gcs_uri, mime_type="video/webm"),
        MOTION_ANALYSIS_PROMPT, # detailed prompt template
    ],
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        thinking_config=types.ThinkingConfig(thinking_budget=1024),
    ),
)
analysis = json.loads(response.text)
```

第 3 階段：生成 Nano Banana 虛擬化身

Gemini Flash Image (`gemini-3.1-flash-image-preview`) 會使用從影片擷取的最佳畫面，生成 1024×1024 的風格化虛擬人偶。

為什麼是「Nano Banana」？Nano Banana 是 Gemini 原生圖像生成功能的內部名稱。這項工具會根據參考相片和風格提示 (例如「像素英雄」、「賽博新星」)，生成 PNG 格式的高畫質風格化虛擬人偶。

運作方式 (backend/app/services/nano_banana_service.py) :

```
# Simplified from nano_banana_service.py
response = client.models.generate_content(
    model="gemini-3.1-flash-image-preview",
    contents=[
        types.Content(role="user", parts=[
            types.Part.from_bytes(data=frame_bytes, mime_type="image/png"),
            types.Part.from_text(text=avatar_prompt),
        ])
    ],
    config=types.GenerateContentConfig(
        response_modalities=["IMAGE"],
        image_config=types.ImageConfig(
            aspect_ratio="1:1",
            output_mime_type="image/png",
        ),
    ),
)
```

系統會將生成的虛擬人偶 PNG 上傳至 GCS，並傳遞至下一個階段。

第 4 階段：生成 Veo 影片

系統會將大頭貼圖片做為 **Veo 3.1** (`veo-3.1-fast-generate-001`) 的**參考素材**，生成 8 秒的 AI 影片。

特別之處： Veo 會以 Gemini 製作的 `veo_prompt` (描述動作、節奏和風格) 和虛擬人偶做為參考圖像，生成虛擬人偶做出類似動作的全新影片。

運作方式 (`backend/app/services/veo_service.py`)：

```
# Simplified from veo_service.py
config = GenerateVideosConfig(
    reference_images=[
        VideoGenerationReferenceImage(
            image=Image(gcs_uri=avatar_gcs_uri, mime_type="image/png"),
            reference_type="ASSET",
        )
    ],
    aspect_ratio="16:9",
    duration_seconds=8,
    output_gcs_uri=f"gs://{BUCKET}/output/{video_id}/",
)
operation = client.models.generate_videos(
    model="veo-3.1-fast-generate-001",
    prompt=veo_prompt,
    config=config,
)
```

Veo 生成作業是**非同步**作業，會立即傳回作業 ID。後端會輪詢作業，直到完成為止 (最多 10 分鐘)。

第 5 階段：後續處理管道

Veo 完成後，**背景管道** (`backend/app/services/pipeline.py`) 會自動執行：

1. 將 8 秒的 Veo 輸出內容**剪輯**為 3 秒
2. **合成並排影片** (左側為原始錄製內容，右側為 AI 影片)
3. 上傳合成影片至 GCS
4. 釋放佇列位置

這個管道會以背景 `asyncio.Task` 執行，資訊站前端不必等待。

佇列系統

由於生成 Veo 影片需要大量資源，系統會**強制執行最多 3 項並行工作**：


```
# backend/app/routers/queue.py
MAX_CONCURRENT_JOBS = 3

@router.get("/queue/status")
async def queue_status():
    return {
        "active_jobs": len(_active_jobs),
        "max_jobs": MAX_CONCURRENT_JOBS,
        "available": len(_active_jobs) < MAX_CONCURRENT_JOBS,
    }
```

前端會先檢查 `GET /api/queue/status`，再允許新使用者啟動工作階段。管道完成並呼叫 `complete(video_id)` 時，系統會為下一個使用者開啟時段。

Cloud Run - 無伺服器容器

後端和前端都會部署為 **Cloud Run 服務**：

為何選擇 Cloud Run？

- **無需管理伺服器**：Google 會處理擴充、修補和硬體事宜
- **調度資源率降至零**：只有在要求處於啟用狀態時才需付費
- **以容器為基礎**：使用 Dockerfile 部署任何語言或架構
- **內建 HTTPS**：每項服務都會獲得永久的安全網址
- **自動調整資源配置**：自動處理尖峰流量 (本示範中為 1 到 3 個執行個體)

服務	目的	金鑰設定
後端	FastAPI API 伺服器	2 GiB 記憶體 (用於透過 ffmpeg 處理影片)
前端	由 Nginx 提供的靜態 React 應用程式	預設記憶體容量

步驟 5 · 約 0 分鐘

5. 執行設定指令碼

1. 執行自動設定

`setup.sh` 指令碼會建立必要的雲端資源，並產生 `.env` 檔案。

👉  將指令碼設為可執行狀態並運作執行：

```
cd ~/gemini-motion-lab-starter
chmod +x setup.sh
./setup.sh
```

setup.sh 的用途：

1. 為後端建立服務帳戶 (`gemini-motion-lab-sa`)，以便存取 GCS 和 Vertex AI
2. 建立 **GCS bucket** (`gemini-motion-lab-${PROJECT_ID}`)，並設定 CORS 以供瀏覽器上傳
3. 產生 `.env` 檔案，並預先填入所有必要環境變數

2. 授予 IAM 角色

現在請將必要權限授予服務帳戶。

👉  執行下列指令，設定專案 ID 並授予所有三個角色：

```
export PROJECT_ID=$(cat ~/project_id.txt)

# 1. Storage Admin – upload/download videos and frames
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:gemini-motion-lab-sa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/storage.admin"

# 2. Vertex AI User – call Gemini and Veo models
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:gemini-motion-lab-sa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"

# 3. Service Account Token Creator – generate signed URLs for GCS
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")
COMPUTE_SA="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"

gcloud iam service-accounts add-iam-policy-binding \
  gemini-motion-lab-sa@${PROJECT_ID}.iam.gserviceaccount.com \
  --project=$PROJECT_ID \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/iam.serviceAccountTokenCreator"
```

各個角色的權限：

角色	目的
roles/storage.admin	具備在 Cloud Storage 中讀取、寫入及管理物件的完整權限，這是上傳影片、影格和合成輸出內容的必要條件
roles/aiplatform.user	使用 Vertex AI 模型進行預測，這是 Gemini 分析、Nano Banana 虛擬人偶生成和 Veo 影片製作的必要條件
roles/iam.serviceAccountTokenCreator	產生已簽署的網址，讓分享頁面可透過有時間限制的連結，將影片提供給行動裝置

3. 驗證 .env 檔案

👉🖥️ 檢查產生的 .env 檔案：

```
cat .env
```

如下所示：

```
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
GCS_BUCKET=gemini-motion-lab-your-project-id
GCS_SIGNING_SA=gemini-motion-lab-sa@your-project-id.iam.gserviceaccount.com
GOOGLE_GENAI_USE_VERTEXAI=true
MOCK_AI=false
```

步驟 6 · 約 0 分鐘

6. 🚀 部署後端

1. 瞭解後端 Dockerfile

部署前，請先瞭解容器的外觀：

```
# backend/Dockerfile
FROM python:3.11-slim                    # Python base image
RUN apt-get update && apt-get install -y \
    ffmpeg libgll libglib2.0-0 \        # ffmpeg for video processing
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir .        # Install Python dependencies
COPY app/ ./app/                        # Copy application code
EXPOSE 8080
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

為什麼 **ffmpeg**？後端會使用 **ffmpeg** 執行下列操作：

- 從上傳的影片中擷取最佳畫面 (用於生成虛擬化身)
- 將 Veo 輸出內容從 8 秒剪輯為 3 秒
- 製作並排比較影片 (原始影片 + AI 影片)

因此後端需要 `--memory 2Gi`，因為影片處理作業會耗用大量記憶體。

2. 部署至 Cloud Run

👉 📁 載入環境變數並部署：

```
source .env

cd ~/gemini-motion-lab-starter/backend

gcloud run deploy gemini-motion-lab-backend \
  --source . \
  --region us-central1 \
  --allow-unauthenticated \
  --min-instances 1 \
  --max-instances 3 \
  --memory 2Gi \
  --port 8080 \
  --project $GOOGLE_CLOUD_PROJECT \
  --set-env-vars
"GOOGLE_CLOUD_PROJECT=$GOOGLE_CLOUD_PROJECT,GOOGLE_CLOUD_LOCATION=$GOOGLE_CLOUD_LOCATION,GCS_
BUCKET=$GCS_BUCKET,GCS_SIGNING_SA=$GCS_SIGNING_SA,GOOGLE_GENAI_USE_VERTEXAI=$GOOGLE_GENAI_USE
_VERTEXAI,MOCK_AI=$MOCK_AI"
```

主要部署旗標說明：

檢舉	目的
<code>--source .</code>	直接從來源建構 Docker 映像檔 (由 Cloud Build 處理)
<code>--allow-unauthenticated</code>	公開 API (前端必須這麼做)
<code>--min-instances 1</code>	至少保持 1 個執行個體處於暖機狀態，避免冷啟動
<code>--max-instances 3</code>	限制縮放 (因為我們也限制最多 3 項並行 Veo 工作)
<code>--memory 2Gi</code>	使用 ffmpeg 處理影片時必須提供
<code>--set-env-vars</code>	將所有設定做為環境變數傳遞

這項作業約耗時 3 至 5 分鐘。Cloud Build 會執行下列操作：

1. 上傳原始碼
2. 建構 Docker 映像檔
3. 將其推送至 Artifact Registry
4. 將其部署至 Cloud Run

3. 儲存後端網址

👉🏻 部署完成後，請儲存後端網址：

```
BACKEND_URL=$(gcloud run services describe gemini-motion-lab-backend \
  --region us-central1 \
  --format="value(status.url)" \
  --project $GOOGLE_CLOUD_PROJECT)

echo "Backend URL: $BACKEND_URL"
```

4. 更新後端分享網址

後端會產生 QR code，方便使用者下載影片。為此，應用程式需要知道自己的公開網址。

👉🏻 使用專屬網址更新後端設定：

```
gcloud run services update gemini-motion-lab-backend \  
  --region us-central1 \  
  --update-env-vars PUBLIC_BASE_URL=$BACKEND_URL \  
  --project $GOOGLE_CLOUD_PROJECT
```

5. 驗證後端

👉 🖥️ 測試健康狀態端點：

```
curl $BACKEND_URL/api/health
```

預期的輸出內容：

```
{"status":"ok"}
```

👉 🖥️ 查看佇列狀態：

```
curl $BACKEND_URL/api/queue/status
```

預期的輸出內容：

```
{"active_jobs":0,"max_jobs":3,"available":true}
```

🎉 您的後端已上線！現在可以上傳影片、執行 Gemini 分析、生成虛擬化身，以及製作 Veo 影片。

步驟 7 · 約 0 分鐘

7. 🎨 部署前端

1. 瞭解前端 Dockerfile

前端使用多階段建構，先建構 React 應用程式，然後透過 Nginx 提供服務：

```
# frontend/Dockerfile
FROM node:20-alpine AS builder                                # Stage 1: Build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE=https://...                                # Backend URL baked at build time
ENV VITE_API_BASE=$VITE_API_BASE
RUN npm run build                                              # Produces static files in /app/dist

FROM nginx:alpine                                             # Stage 2: Serve
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 8080
```

為什麼 `ARG VITE_API_BASE`？Vite 是建構時期的工具，環境變數必須在**建構時間**注入，而非執行階段。`ARG` 指令可讓我們在 `docker build` 期間傳遞後端網址，而 Vite 會將該網址併入 JavaScript 組合。

也就是說，如果後端網址有任何變更，您必須**重新部署前端**，才能取得新網址。

2. 部署至 Cloud Run

👉 📁 首先，將後端網址寫入 `.env` 檔案，讓 Vite 在建構時間將其烘焙進去：

```
cd ~/gemini-motion-lab-starter/frontend
echo "VITE_API_BASE=$BACKEND_URL" > .env
```

👉 📁 現在部署前端：

```
gcloud run deploy gemini-motion-lab-frontend \
  --source . \
  --region us-central1 \
  --allow-unauthenticated \
  --min-instances 1 \
  --max-instances 3 \
  --port 8080 \
  --project $GOOGLE_CLOUD_PROJECT
```

這個過程大約需要 2 到 3 分鐘。

3. 取得前端網址

👉 🖥️ 擷取並開啟前端網址：

```
FRONTEND_URL=$(gcloud run services describe gemini-motion-lab-frontend \  
  --region us-central1 \  
  --format="value(status.url)" \  
  --project $GOOGLE_CLOUD_PROJECT)  
  
echo "🎬 Your Gemini Motion Lab is live at: $FRONTEND_URL"
```

👉 在瀏覽器中開啟網址，應該會看到 Gemini Motion Lab 資訊亭介面！

🎉 兩項服務都已部署完成！前端會透過內建的 `VITE_API_BASE` 網址呼叫後端。Cloud Run 服務網址是永久的，重新部署時不會變更。

8. 🎮 [選用] 試玩試用版

1. 錄製動態相片

- 1. 在瀏覽器中開啟「前端網址」（建議使用 Chrome，可獲得最佳攝影機支援）
- 2. 按一下「開始」即可開始錄製
- 3. 跳舞或移動約 5 秒，大幅度擺動手臂和動態姿勢效果最佳
- 4. 系統會自動停止錄製並上傳檔案

2. 觀看 AI 管道

上傳後，您會即時看到管道執行情況：

階段	發生什麼事	時間長度
分析中...	Gemini Flash 會分析影片中的動作模式	~5-10 秒
正在生成虛擬化身...	Nano Banana 會從最佳畫面製作風格化虛擬人偶	最多 8 到 12 秒
正在製作影片...	Veo 3.1 會根據虛擬化身和動作提示生成 AI 影片	約 60 到 120 秒
撰寫中...	ffmpeg 會修剪影片並建立並排比較畫面	~5-10 秒

首次生成時，模型需要初始化，因此可能需要較長時間。後續執行通常會更快。

3. 分享你的創作

管道完成後：

- 1. 資訊亭螢幕上會顯示 **QR code**
- 2. 使用手機掃描 **QR code**
- 3. 系統會顯示針對行動裝置最佳化的分享頁面，當中包含您製作的影片

4. 檢查後端記錄

👉📺 查看幕後發生了什麼事：

```
gcloud logging read \  
  "resource.type=cloud_run_revision AND resource.labels.service_name=gemini-motion-lab-  
backend" \  
  --limit=30 \  
  --project $GOOGLE_CLOUD_PROJECT \  
  --format="value(timestamp,textPayload)" \  
  --freshness=10m
```

您會看到追蹤管道的記錄行：

```
Pipeline started for video_id=abc123  
Gemini model used: gemini-3-flash-preview  
Avatar generated: style=pixel-hero size=450KB time=8.2s  
Veo model used: veo-3.1-fast-generate-001  
Pipeline: Veo complete for video_id=abc123  
Pipeline: trimmed video uploaded  
Pipeline: composed video uploaded  
Pipeline complete for video_id=abc123
```

5. 監控佇列

👉🏻 檢查正在執行的工作數量：

```
curl $BACKEND_URL/api/queue/status
```

如果同時有 3 個工作階段處於啟用狀態，回應會顯示：

```
{"active_jobs":3,"max_jobs":3,"available":false}
```

新使用者必須等待空位出現。

步驟 9 · 約 0 分鐘

9. 🎉 結論

建構項目

- ✅ **AI 動作分析**：Gemini Flash 會分析影片的動作、節奏和風格
- ✅ **生成虛擬化身**：Nano Banana 會從影片影格生成風格化的虛擬化身
- ✅ **AI 影片製作**：Veo 3.1 會生成符合使用者動作的新影片
- ✅ **非同步管道**：透過佇列管理進行背景處理 (最多 3 個並行)
- ✅ **並排合成**：以 ffmpeg 為基礎的影片合成功能
- ✅ **Cloud Run 部署作業**：無伺服器、自動調整資源配置、免除伺服器管理作業

學到的重要概念

1. **Gemini Multimodal**：傳送影片做為輸入內容，並接收結構化 JSON 分析結果
2. **Nano Banana (Gemini 圖像生成)**：使用參考圖像和風格提示生成虛擬替身
3. **Veo 3.1**：根據參考素材和文字提示詞非同步生成影片
4. **Cloud Run**：部署含有環境變數和自動調整資源配置功能的容器
5. **非同步管道模式** - 使用 `asyncio.Task` 啟動後不受控的背景工作，執行長時間的 AI 作業
6. **佇列管理**：限制並行 AI 工作數量，控管費用和 API 配額

架構回顧

AI Dance Avatar Video Pipeline



後續步驟

- 新增更多顯示圖片樣式：編輯 `backend/app/prompts/avatar_generation.py`
- 自訂 **Veo** 提示：編輯 `backend/app/prompts/video_generation.py`
- 在本機以模擬模式執行 - 在 `.env` 中設定 `MOCK_AI=true`，即可進行開發，不必呼叫 API
- 擴大活動規模：增加 `--max-instances` 和 `MAX_CONCURRENT_JOBS`

資源

- [Gemini API 說明文件](#)
- [Veo API 說明文件](#)

- [Cloud Run 說明文件](#)
- [FastAPI 說明文件](#)

恭喜！您已在 Cloud Run 上部署正式版 AI 管道，將 Gemini、Nano Banana 和 Veo 串連在一起。同樣的非同步管道模式適用於任何多步驟 AI 工作流程，包括內容生成、資料處理和創意工具。
